top КОЛЛЕДЖ

Межпроцессное взаимодействие

Операционные системы и среды

top КОЛЛЕДЖ

О межпроцессном взаимодействии

Почему оно необходимо



Вопрос

top КОЛЛЕДЖ

Почему процессы не могут взаимодействовать друг с другом напрямую?

Вспоминаем о процессах

top КОЛЛЕДЖ

- Одна из задач процесса — изолировать свою собственную память от вмешательства извне.
- Решая проблему разделенного доступа к оперативной памяти, мы создали новую — сложность коммуникации между процессами.



Межпроцессное взаимодействие

top КОЛЛЕДЖ

- ***Межпроцессное взаимодействие*** (inter-process communication, IPC) — механизм обмена данных между двумя и более процессами.



top КОЛЛЕДЖ

Способы межпроцессного взаимодействия

Известные и новые

Способы взаимодействия

top КОЛЛЕДЖ

- Принцип всех способов межпроцессного взаимодействия *один и тот же*: раз мы изолировали память нескольких процессов, то нужно создать некоторую ***дополнительную буферную зону***, в которой будет осуществляется совместный доступ к данным несколькими процессами.





Вопрос

top КОЛЛЕДЖ

Назовите на ваш взгляд самый простой буфер для обмена данными между процессами.

Буферный файл

top КОЛЛЕДЖ

- Все ОС так или иначе взаимодействуют с логическим представлением данных в виде ***файла***.
- Так как любой процесс имеет доступ к файловой системе благодаря интерфейсу системных вызовов, созданный на ВЗУ файл можно использовать в качестве буферной зоны для обмена информацией между процессами.



Существенные недостатки

top КОЛЛЕДЖ

- Информацию из файла использовать небезопасно: она может быть изменена извне пользователем, или файл может быть удален
  - ? Решение: использовать политики безопасности (ACL или атрибуты файловой системы) для данного файла
- Сложно осуществлять контроль за изменением содержимого — нужно постоянно проверять файл на наличие изменений
  - ? Решение: создание прерывания или другой внешней функции контроля (?)
- Скорость обмена информацией ограничена скоростью записи и считывания ВЗУ
  - ? Решение: использовать ОЗУ или другую быструю память



Вопрос

top КОЛЛЕДЖ

С какими видами межпроцессного взаимодействия мы уже сталкивались?

Подсказка: вспомните о завершении процесса

top КОЛЛЕДЖ

Сигналы

Посылаем сигналы процессам

Сигналы

top КОЛЛЕДЖ

- Процесс может послать другому процессу ***сигнал*** — асинхронное уведомление о каком-либо событии.
- Сигналы являются частью стандарта POSIX, поэтому присутствует не во всех ОС. ~~(привет, Windows)~~
- В некоторых случаях по умолчанию получение сигнала ведет к завершению процесса.



Сигналы

top КОЛЛЕДЖ

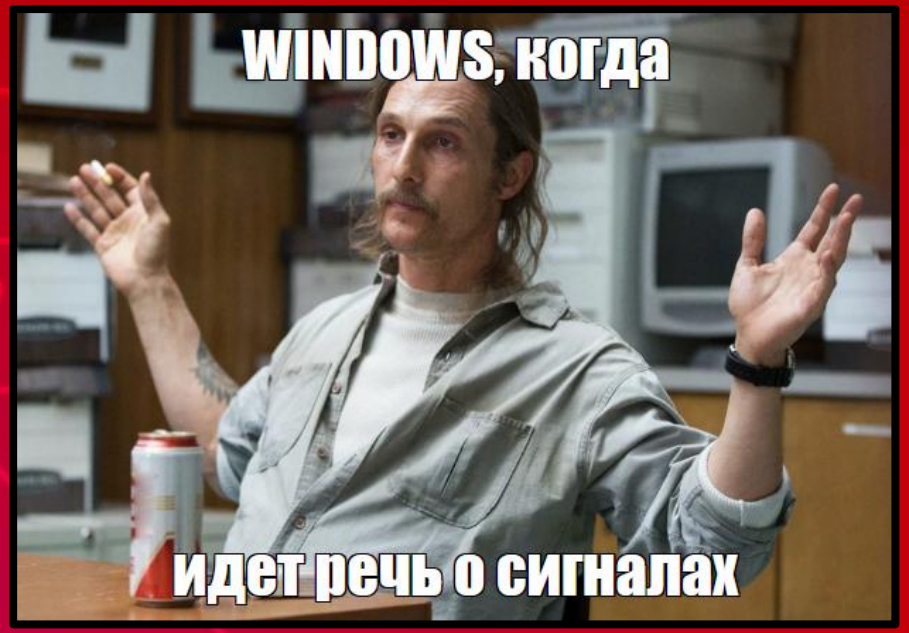
- Копия слайда с Темы №3 с дополнением.
- В случае системы Unix внешний сигнал (утилита `kill`) необязательно завершает процесс. Сигналов существует множество:
  - `SIGHUP` (hang up) — процесс потерял соединение с терминалом
  - `SIGINT` (interrupt) — остановка процесса вводом Ctrl + C
  - `SIGSTOP` — принудительная остановка, однако не завершает процесс
  - `SIGTERM` (terminate) — отправка запроса на завершение процесса
  - `SIGKILL` — принудительное завершение процесса
  - `SIGPWR` — сигнал об аварии сетевого питания
  - и другие.

☺ А также пользовательские: SIGUSR1 и SIGUSR2

Сигналы

top КОЛЛЕДЖ

- *Как сигнал послать?* (С) Студенты
- В Windows — никак, за исключением POSIX-совместимых системных сигналов:
  - `SIGINT, SIGTERM, SIGALRM`
- В Unix — используется команды `kill` и `pkill` (или аналогичный системный вызов) с именем сигнала в качестве первого аргумента.
  - `pkill -SIGUSR1 process`



Сигналы

top КОЛЛЕДЖ

- *Что можно с помощью сигналов делать, кроме завершения процесса?* (С) Студенты
- Пользовательским сигналам можно задать свое собственное поведение. В коде программы можно перехватить сигнал и задать ему исполняемую функцию с помощью системного вызова `signal`.
- Более того, собственное поведение можно задать для большей части сигналов (если точнее, то всем, кроме `SIGKILL`, `SIGSTOP` и `SIGCONT`, а также `SIGRTMIN` и `SIGRTMIN+1` из POSIX Threads).

Демонстрация примера

top КОЛЛЕДЖ



top КОЛЛЕДЖ

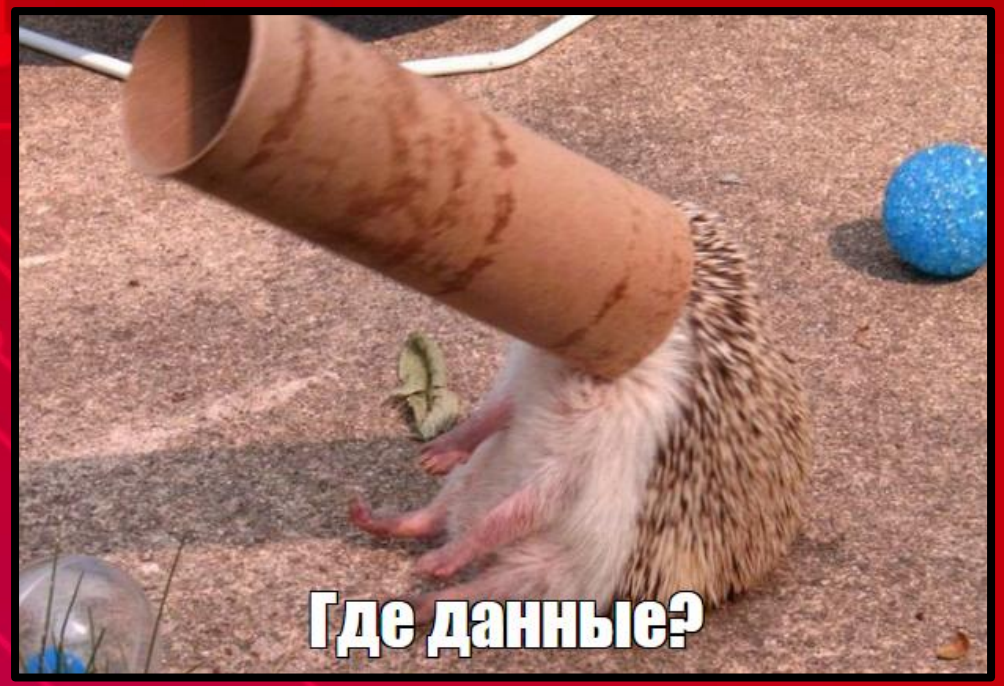
Каналы

Неименованные, именованные

Каналы

top КОЛЛЕДЖ

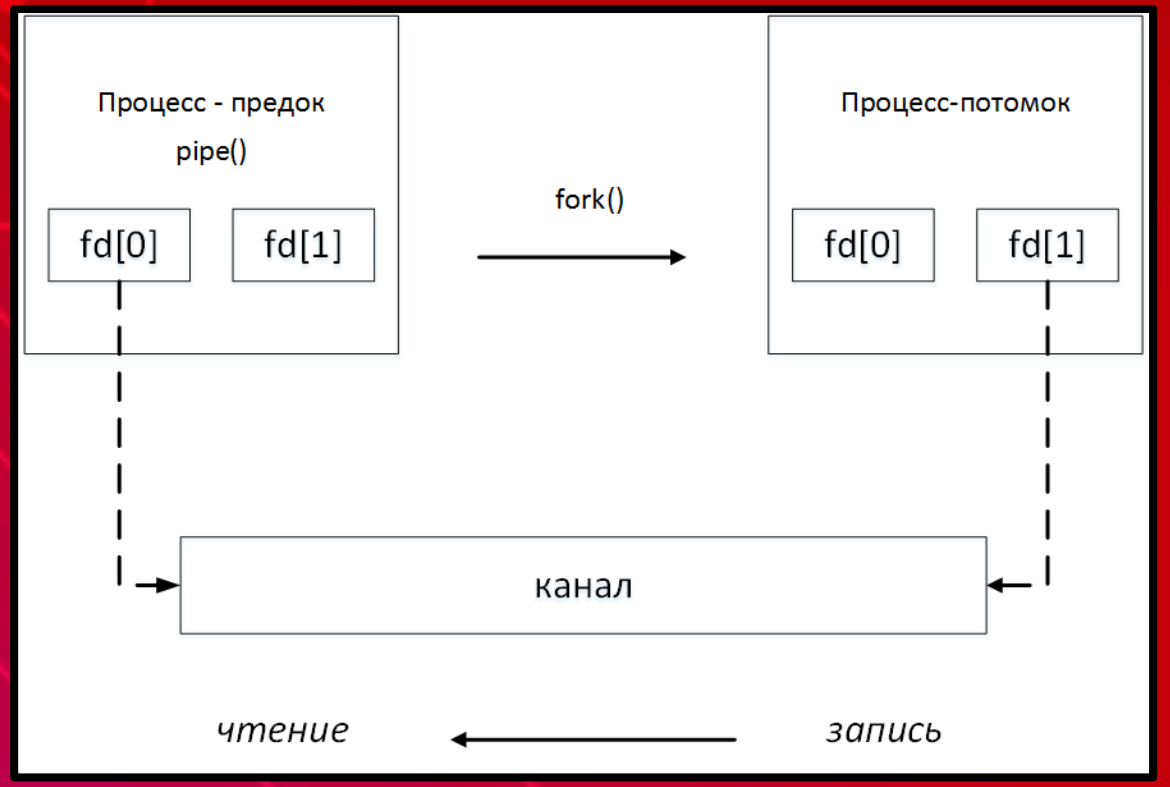
- ***Канал*** (иногда, ***конвейер***, англ. pipeline) — один из элементарных способов взаимодействия процессов, похожий на буферный файл.
- При создании канала в системе создается специализированный буфер, который имеет файловые дескрипторы для чтения и записи данных.



Каналы

top КОЛЛЕДЖ

- Канал используется для передачи данных между родительским и дочерними процессами.
- Канал может быть создан только родительским процессом.
- Из этого следует вывод, что в Windows понятия канала также не существует из-за отсутствия иерархии процессов.



Демонстрация примера

top КОЛЛЕДЖ





Вопрос

top КОЛЛЕДЖ

Как решить проблему того, что к каналу имеют доступ только родительский процесс и его потомки?

Именованный канал

top КОЛЛЕДЖ

- ***Именованный канал*** (англ. named pipe, ***FIFO***) — канал, который имеет уникальное символьное имя, по которому могут обращаться произвольные процессы.
- Именованные каналы реализованы как в POSIX, так и в WinAPI.
- Данные в каналах отсылаются и принимаются в виде потока байтов.



Демонстрация примера

top КОЛЛЕДЖ



top КОЛЛЕДЖ

Сообщения

Расширенный функционал сигнала

Обмен сообщениями

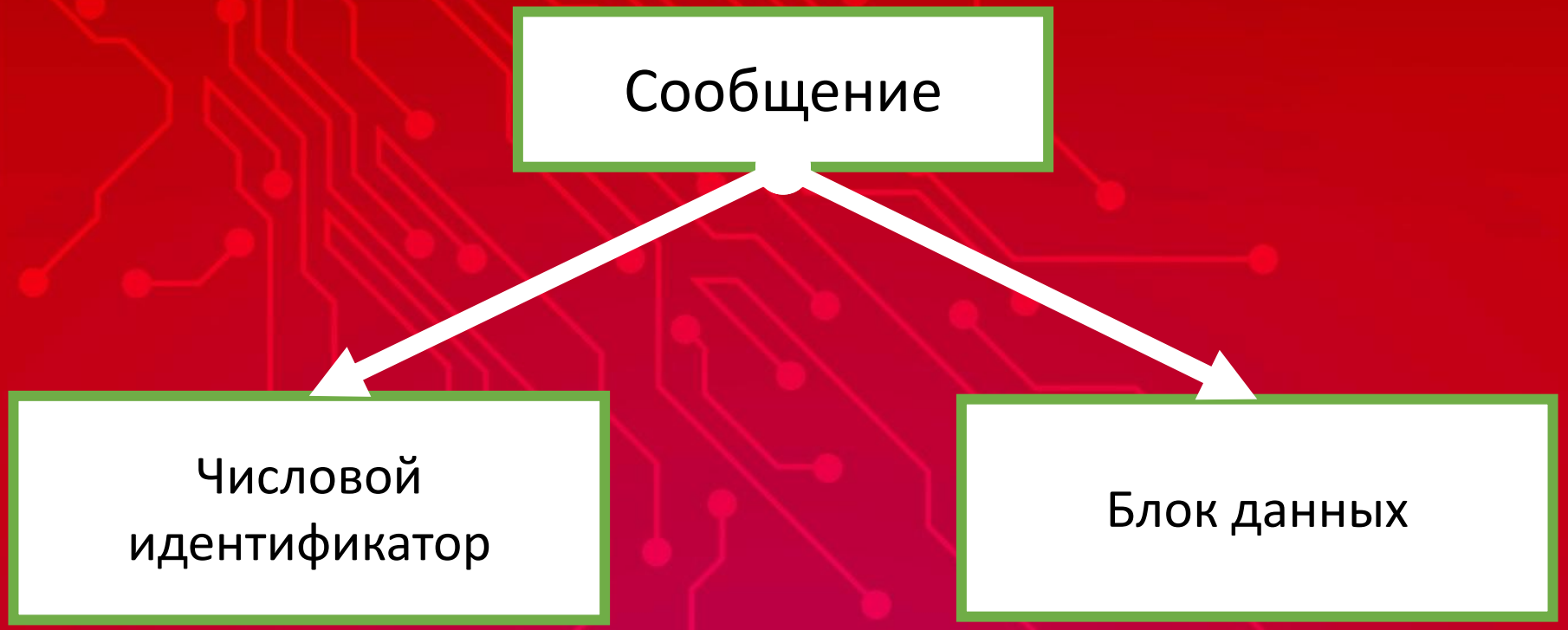
top КОЛЛЕДЖ

- Сигналы не подходят для передачи осмысленных данных, хотя позволяют передать информацию о состоянии. Этого часто недостаточно для полноценного взаимодействия между процессами.
- Данный недостаток исправляется концепцией ***сообщений***.



Обмен сообщениями

top КОЛЛЕДЖ



Обмен сообщениями

top КОЛЛЕДЖ

- В основе обмена сообщениями лежит очередь, в которую процессы накапливают запросы. Для добавления в очередь используется функция `msgsnd`.
- Для получения данных, сообщение должно быть изъято из очереди. Для этого используется функция `msgget`.
- Отличие от сигнала заключается еще и в том, что получение сообщения является синхронным, в отличие от асинхронного сигнала.



Обмен сообщения

top КОЛЛЕДЖ

- Пример популярного набора библиотек для передачи сообщений между процессами — MPI (Message Passing Interface).
- Данные библиотеки используются для параллельных вычислений на основе разделения задачи на множество процессов, работающих со своими копиями данных.



top КОЛЛЕДЖ

Разделяемая память

Отображение памяти в адресное пространство процесса

shmem

Разделяемая память

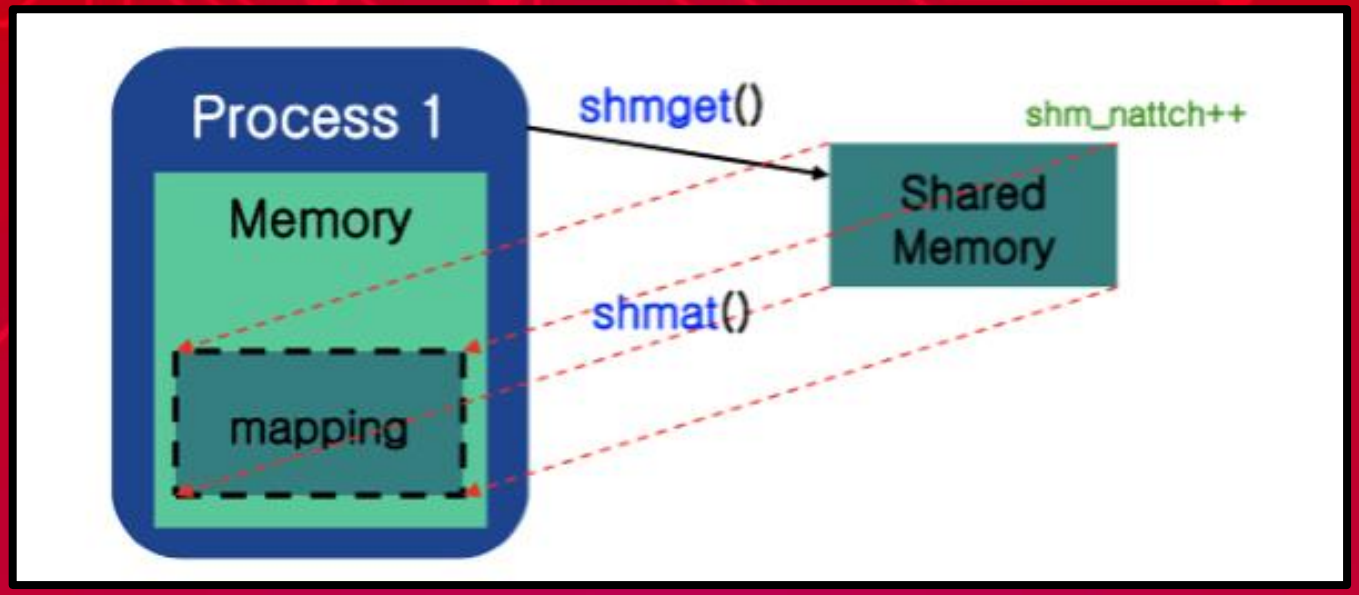
top КОЛЛЕДЖ

- Несмотря на то, что процессы изолируют друг от друга логическое адресное пространство, операционная система не мешает процессу отобразить часть логического адресного пространства на произвольный блок физической памяти, если используется соответствующий уровень привилегий.
- Следовательно, для некоторых процессов можно часть логического адресного пространства отобразить на общее физическое адресное пространство, которое называется ***разделяемой памятью*** (англ. *shared memory*).
- Сделать это можно с помощью ***объекта*** или ***сегмента***.

Разделяемая память (объект)

top КОЛЛЕДЖ

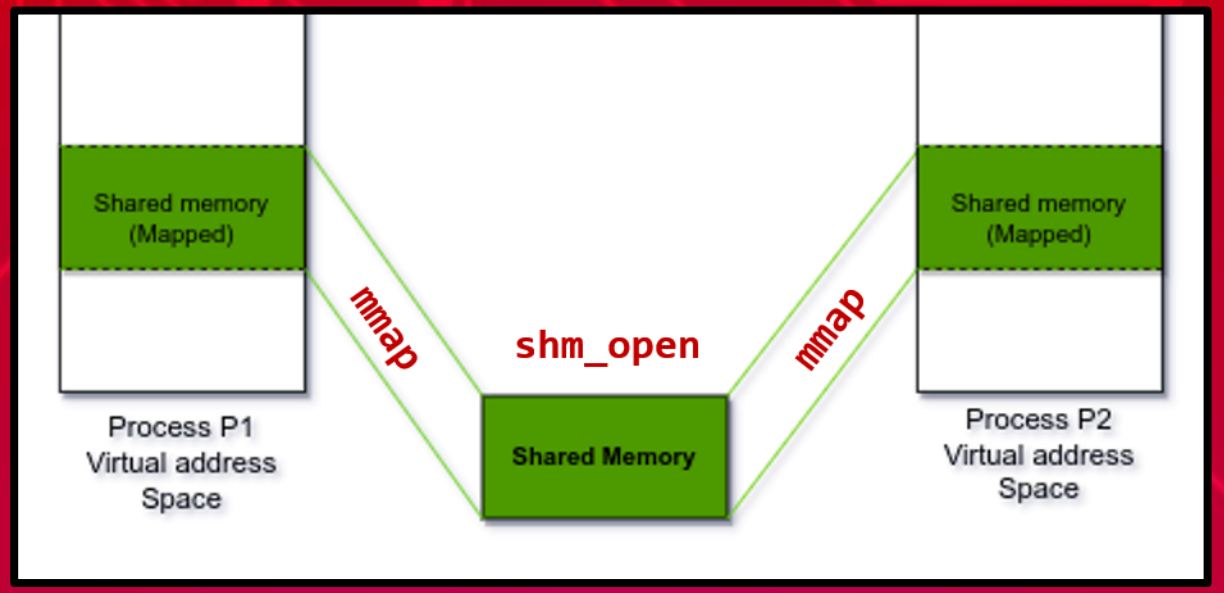
- Системные вызовы Unix для управления разделяемой памятью:
  - `shmget()` — создание объекта и выделение участка разделяемой памяти;
  - `shmat()` — присоединение участка памяти в адресное пространство процесса;
  - `shmdt()` — отсоединение участка памяти от адресного пространства процесса.



Разделяемая память (сегмент)

top КОЛЛЕДЖ

- Системные вызовы Unix для управления разделяемой памятью:
  - `shm_open()` — выделение сегмента разделяемой памяти;
  - `mmap()` — отображение сегмента памяти в адресное пространство процесса;
  - `shm_unlink()` — удаление сегмента разделяемой памяти.



Демонстрация примера

top КОЛЛЕДЖ



top КОЛЛЕДЖ

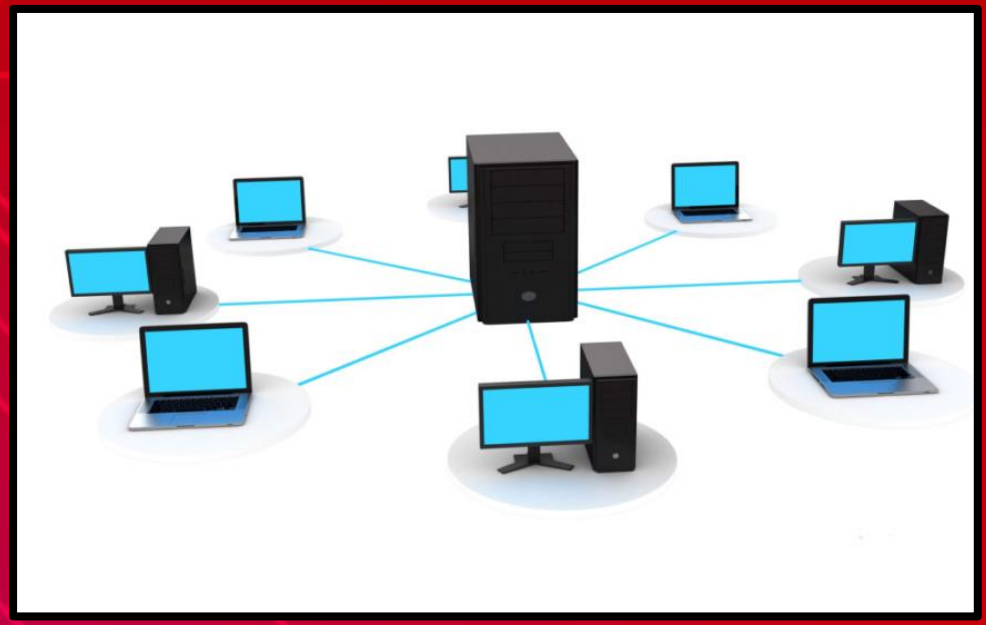
Сокеты

Прокаченные именованные каналы

Сокеты

top КОЛЛЕДЖ

- ***Сокет*** (редко, гнездо, англ. *socket*) — абстрактный объект, представляющий собой конечную точку некоторого подключения, которое определяется в коммуникационный домен, по определенному типу.
- Сокеты используются не только для организации IPC, но и для обмена данными в сетевой инфраструктуре.

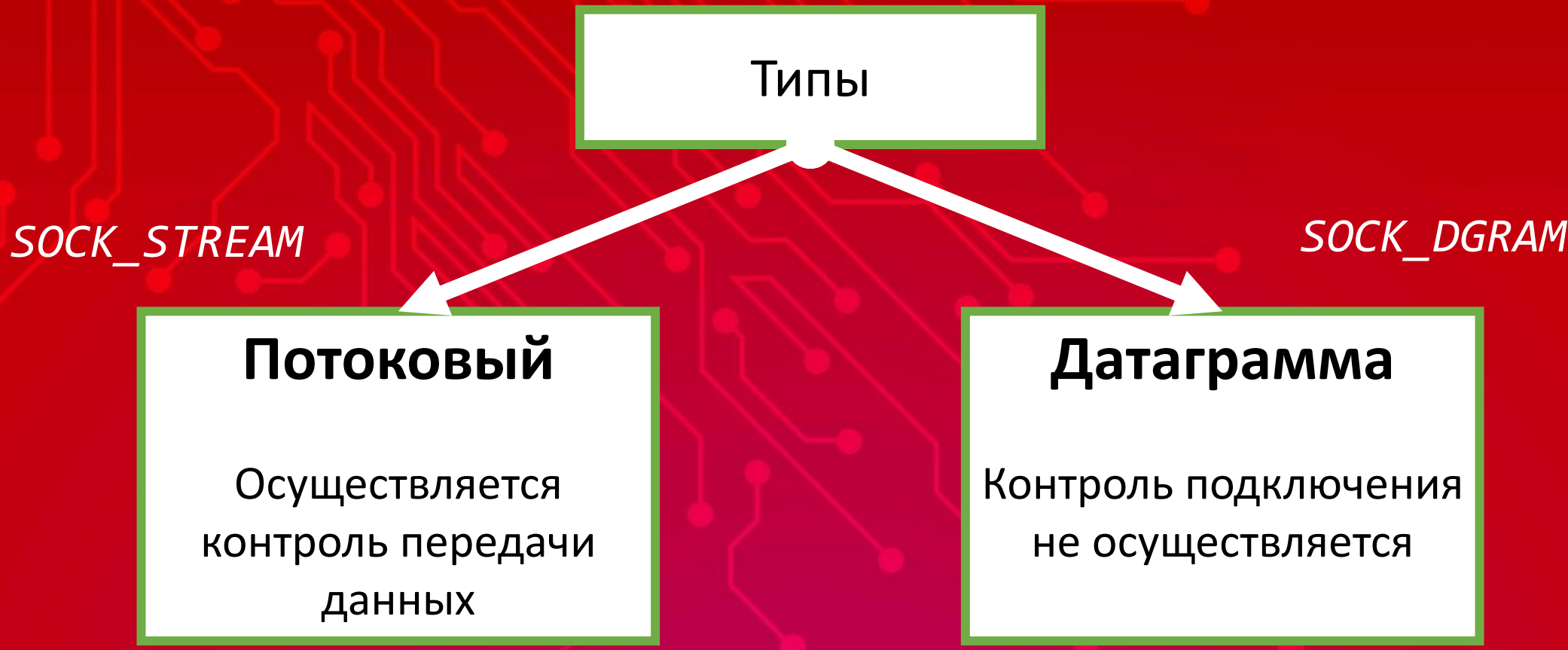


Коммуникационные домены сокетов

top КОЛЛЕДЖ

- В большинстве языков программирования и ОС на данный момент используется реализация сокетов под общим названием ***сокеты Беркли***. Они появились в качестве API в BSD в 1982 году.
- Коммуникационные домены по Беркли:
  - LOCAL — сокеты для организации IPC
  - INET — сокеты для организации сетевого взаимодействия
  - IPX — сокеты для коммуникации по протоколам Novell IPX
  - PACKET — сокеты для взаимодействия интерфейсов устройств

Типы сокетов



Сокеты

top КОЛЛЕДЖ

- Методы для работы с сокетами по POSIX:
  - `socket()` — создание сокета
  - `bind()` — связывание сокета с адресом или файловым дескриптором
  - `connect()` — подключение к сокету по адресу или файловому дескриптору
  - `listen()` — прослушивание сокета, подготовка к принятию соединений
  - `accept()` — принятие запроса сокетом на установление соединения
  - `read()`/`write()` или `send()`/`receive()` — чтение и запись данных
- В языках высокого уровня (например, C#) сокеты, как правило, реализуются на основе протоколов стека TCP/IP и не основываются на спецификации POSIX.
  - Классы `TcpClient`, `TcpListener`, `UdpClient`, `UdpReceiveResult`

Демонстрация примера

top КОЛЛЕДЖ



top КОЛЛЕДЖ

Дополнительные способы взаимодействия

Снова про файлы и немного про безумие

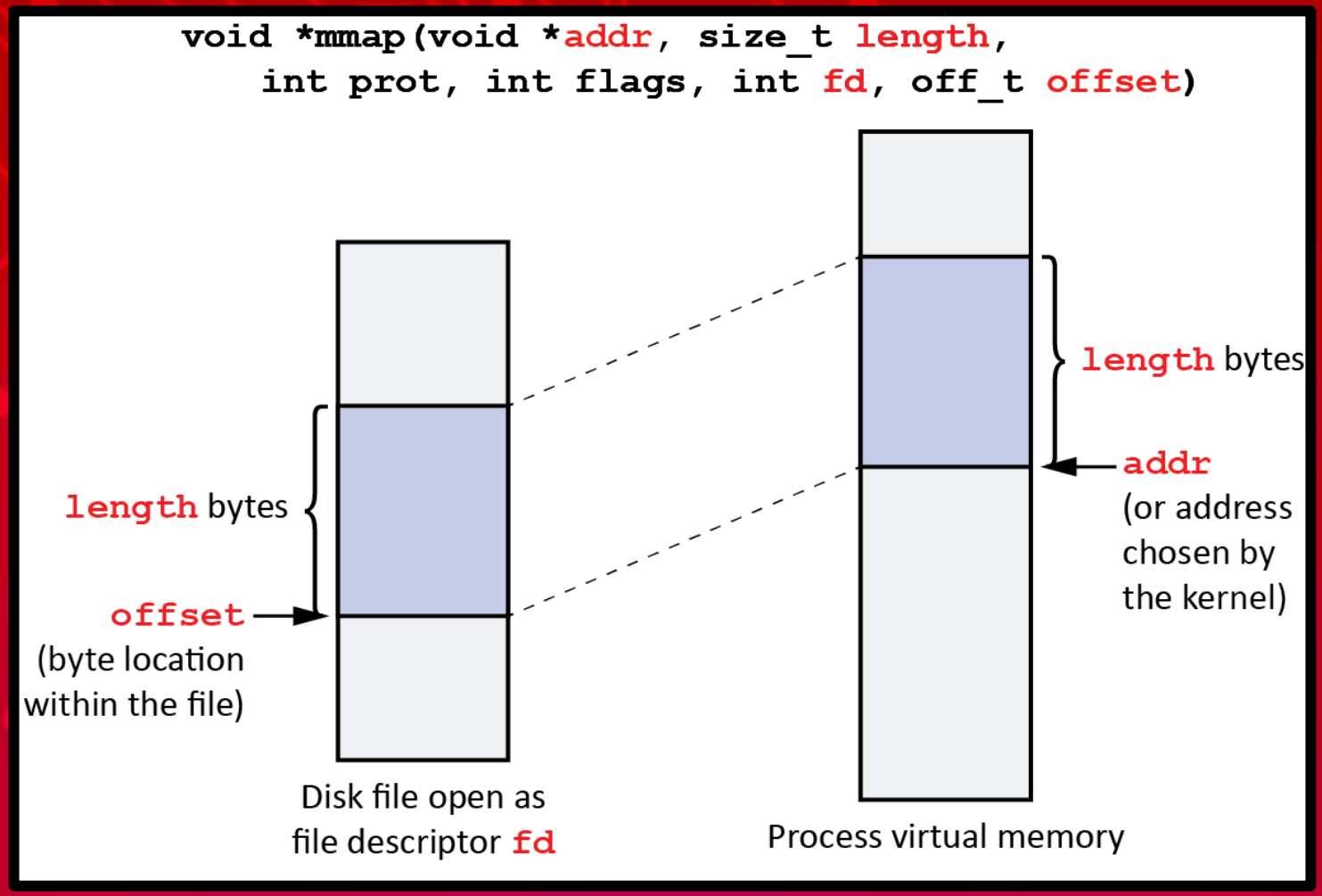
Помните файл?

top КОЛЛЕДЖ

- Оказывается, существует способ отобразить буферный файл в ОЗУ. Это выполняется с помощью системного вызова `mmap` (memory map).
- Действия с данной сущностью похожи на действия с файлом: запись и чтение происходит аналогичным образом, но на уровне скоростей ОЗУ с возможностью контроля изменения данных.
- Впоследствии, данные на ВЗУ синхронизируются с ОЗУ отдельным системным вызовом `fsync`, если нужно.

mmap

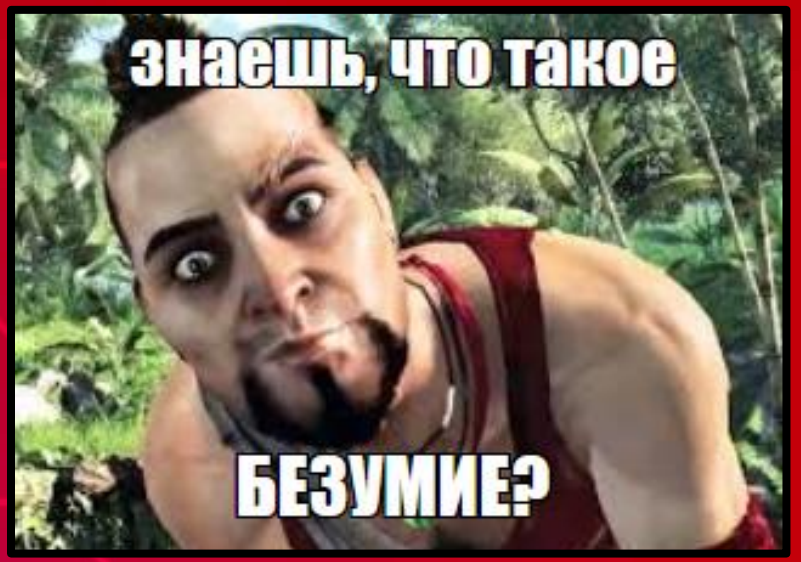
```
void *mmap(void *addr, size_t length,
    int prot, int flags, int fd, off_t offset)
```



Немного о безумном

top КОЛЛЕДЖ

- В некоторых случаях операционные системы позволяют не просто совершать обмен данными между приложениями, но и перехватывать данные и системные вызовы произвольного процесса и даже подменять их.
- В Unix-подобных операционных системах данные операции позволяет совершать системный вызов `ptrace` (***Process Trace***).





Вопрос

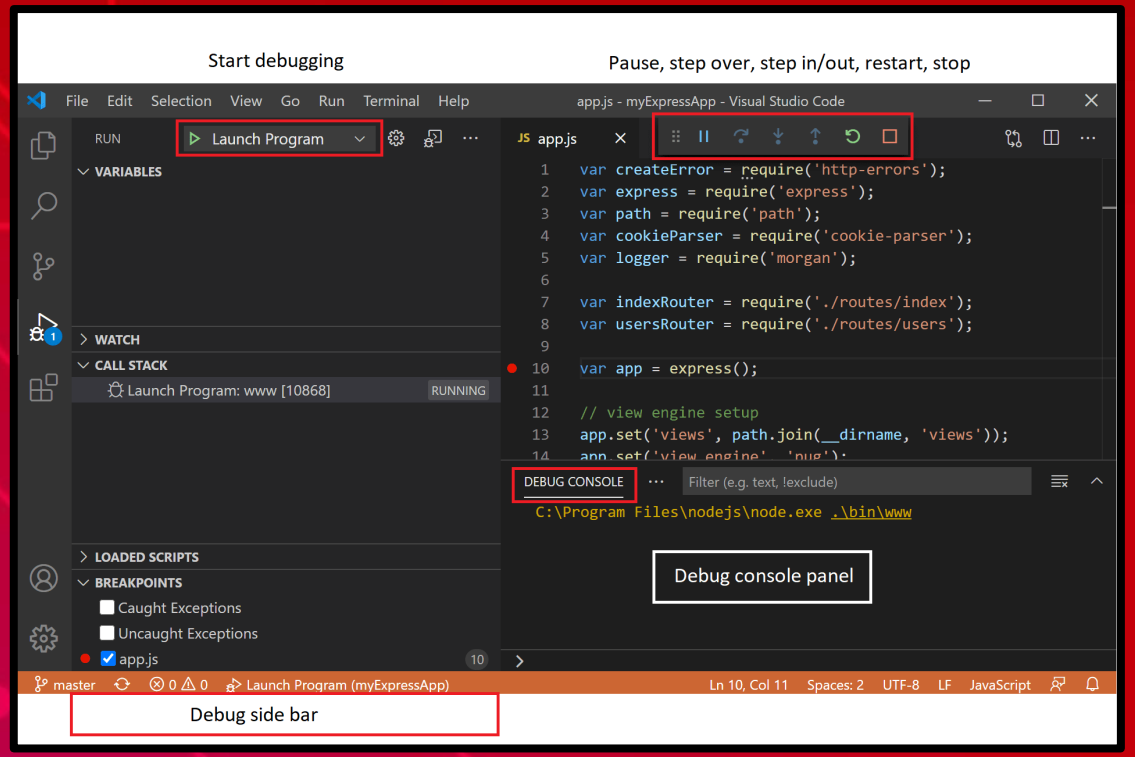
top КОЛЛЕДЖ

Зачем такой механизм может понадобиться?

Немного о безумном

top КОЛЛЕДЖ

- Process Trace используется в ***отладчиках*** (англ. debugger) — данные программы должны в буквальном смысле «владеть» запущенным процессом.
  - обработка системных ошибок;
  - установка точек останова (breakpoint);
  - чтение переменных и регистров;
  - просмотр стека вызовов
  - и т.п.





Вопрос

top КОЛЛЕДЖ

Вопросы?